

Feature: People's time

From build-plan: feature 5a

Branch: feature/05a-peoples-time

Status: verified. Whole feature seen and agreed by Frank 2026-10-01; steps 5a.1 and 5a.2 built, tested and reviewed step by step; every blocking review finding fixed, F-51 last on 2026-10-01; F-52 (P3) carried to 5d. The checkpoint for the final review.

Branch named in the workspace's `feature/NN-name` form, so one number finds the branch, the archive (`05a-peoples-time.md`) and the tag (`item-05a-done`). The first of feature 5's four parts (split 2026-10-01, Frank).

Goal

One table that says when each person and each place is taken, and a database that refuses to let two things take the same one at the same time. Every booking (5d) writes a row per person, and per place when the service needs one, with its buffers inside its time; every stretch of time off writes a row too. The database refuses two overlapping active rows for the same person or place, so two customers can never both take 3pm and no code path can forget the check. A cancelled row stops blocking. No double booking, the owner included (project plan decisions 21 and 27).

With it, the three backend functions every later part uses: hold time (all of a booking's people and places at once, or none), release it, and read who is taken over a stretch of time, which 5c's free times subtract.

Nothing is visible yet. Time off is entered on screen in feature 12b.

In scope

- The `commitment` table and its migration, with the no-overlap rule as a database exclusion constraint, which needs Postgres's standard `btree_gist` add-on switched on in the same migration.
- `holdTime` , `releaseTime` and `findCommitments` in `backend/lib/scheduling/` .
- Tests for every rule and function against the local database, including two holds for the same time at the same instant.

Out of scope

- `lead` and `booking` (5d). `commitment.bookingId` is a plain column here; 5d adds its link to `booking` and the rule that a booking's row names one.
- Who does what, standby (5b). Free times (5c). Writing to Google (5d).
- A screen for time off (12b) and any web route: these are backend functions, as in feature 4.

- Recurring time off, travel time between jobs (named, not planned).
- Applying the migration to Railway (at deploy, Frank's line).

Build loop

`workflow.stepReview` is `every` and `workflow.checkpointCommits` is `enabled`.

Each step's plan (Part 1, what it builds; Part 2, Done when) is gone through with Frank and gets its own yes just before it is built. After that yes the step runs straight through without asking: build, tests and checks, tick the box here, write the build log entry and push buildlogs, commit (`feat: 5a.N <what>`) and push to the feature branch, `/audit` scoped to the step, then the independent review. The one planned stop is after the review, where its findings are talked through. P0/P1 are fixed before the next step, or Frank accepts them with a reason; P2/P3 are recorded and carried.

Only three things stop a step earlier: the agreed plan turns out wrong while building, a line only Frank crosses (a package, Railway or real data, `main`, a merge, a force push, deleting anything), or blocking review findings.

`/complete` runs its own final review over the already-reviewed steps, then merges with a merge commit on Frank's yes.

Build steps

- ☒ **5a.1 The commitments table and its rule.**
 - **Blocker for this step's plan, Frank's yes:** switching on `btree_gist` (see Open questions).
 - `packages/shared/db/scheduling-tables/commitment-table.ts` and migration `0009_commitment`, generated, then hand-written at its end: `create extension if not exists btree_gist` (placed first) and the exclusion constraint, which Drizzle cannot express.
 - **Done when** backend tests prove, by inserting rows directly: two overlapping active rows for one person are refused by the exclusion constraint by name; two people can be taken at the same time; back to back (one ends at 3pm, the next starts at 3pm) is allowed; a cancelled row no longer blocks; a row whose end is not after its start is refused; a row cannot name another business's person; deleting a person who still holds time is refused, while deleting the whole business takes its rows; and `db:migrate` applies cleanly to the local database. The backend builds.
- ☒ **5a.2 Hold, release and read time.**
 - `backend/lib/scheduling/hold-time.ts`: all the people and places of one booking or one stretch of time off, in one transaction: all held, or none and the answer says the time is taken.
 - `backend/lib/scheduling/release-time.ts`: marks rows cancelled, inside the business only.
 - `backend/lib/scheduling/find-commitments.ts`: the active rows for some people and places that touch a stretch of time, for 5c.

- **Done when** backend tests prove: a booking's person and place are held together; if either is taken, neither is held and the answer is "taken"; two holds for the same time at the same instant give one held and one taken; a released hold frees the time; release and find never reach another business's rows; find returns exactly the active rows touching the stretch (not ones that only meet its edge, not cancelled ones); and no error carries more than a safe reason. The backend builds.

Files / areas

- `packages/shared/db/scheduling-tables/` (new), `packages/shared/db/index.ts`
- `packages/shared/migrations/0009_commitment.sql`, generated from `packages/shared` and finished by hand
- `backend/lib/scheduling/` (new): one function per file, tests beside them

Data / contracts

commitment

- `id` (text), `organizationId` (FK organization, cascade).
- `resourceId` (text, not null): FK (`organizationId`, `resourceId`) to `resource(organizationId, id)`, **no action** on delete: a person or place with any row here, cancelled or past included, cannot be deleted (feature 12 decides what deleting one does), while deleting a whole business removes its rows through `organizationId`. Indexed as `commitment_resource_index` for those checks (F-49).
- `kind` (text, not null, check `booking | time_off`).
- `bookingId` (text, nullable): which booking holds it. Its link to `booking` and the rule "a booking row names its booking" arrive with 5d.
- `startsAt`, `endsAt` (`timestampz`, not null), check `endsAt > startsAt`. Half-open, [`startsAt`, `endsAt`): one ending at 3pm and another starting at 3pm do not overlap. A booking's buffers are inside its time (the caller passes the widened range; 5c and 5d own the buffer arithmetic).
- `status` (text, not null, default `active`, check `active | cancelled`).
- `createdAt`, `updatedAt`.
- **The rule:** `exclude using gist ("organizationId" with =, "resourceId" with =, tstzrange("startsAt", "endsAt", '[]') with &&) where (status = 'active')`, named `commitment_no_overlap`. The business is compared too (F-48): without it, a row naming another business's busy person was refused as busy, which tells the caller that person's schedule; with it, that row never meets their time and is refused by `commitment_resource_fk` as not theirs, busy or free. Two inserts that clash at the same instant can deadlock inside Postgres, which cancels one (40P01, F-51); nothing of it was saved, and `holdTime` tries it again.

`holdTime(organizationId, { resourceIds, startsAt, endsAt, kind, bookingId? })`

- One row per resource, one transaction. Answers `{ held: true, ids }` or `{ held: false }` when the exclusion rule refused any of them (Postgres code `23P01`). A deadlock (`40P01`) is tried again, up to three attempts in all; by then the other hold is saved, so the answer is "taken". Anything else is rethrown as `Holding time failed: <safe reason>`.
- `resourceIds` non-empty, without duplicates; times as `Date` s.

releaseTime(organizationId, ids) sets `status = 'cancelled'` on those rows of that business; answers how many it changed.

findCommitments(organizationId, resourceIds, from, to) answers the active rows of those resources whose time overlaps `[from, to)`, ordered by start: `{ id, resourceId, kind, bookingId, startsAt, endsAt }`.

Testing

Backend tests against the local `scheduleads_dev` with the existing guard, each making its own throwaway business, people and places so every test passes on its own (the lesson of F-53 and F-54). Database rules are proved by expecting Postgres to refuse the bad row with the named constraint. Each new test shown able to fail once. No frontend change.

Notes for the AI

- Drizzle has no exclusion constraints: declare the table and its checks in the schema, and add the extension and the exclusion constraint by hand at the end of the generated SQL, as 0004 and 0006 added their backfills. A later `db:generate` must not try to drop it; check that it produces no change after 0009.
- `btree_gist` ships with Postgres as a standard contrib module; it still has to be created in each database. Check `pg_available_extensions` on the local database in 5a.1 and say so if it is missing.
- The time zone never matters here: everything is `timestamptz` instants.

Open questions

- **Switching on `btree_gist`** (5a.1's blocker). The plan's rule, that the database itself refuses overlaps so no code path can forget, needs an exclusion constraint over a person's id and a time range together, and Postgres needs this standard add-on for that. Recommended: switch it on, in the migration; on Railway it is switched on when the migration runs at deploy. The alternative, a check in our own code under a lock, is exactly what the plan rules out.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- `db/scheduling-tables/` is a new tables folder for what features 5a to 5d add; `drizzle.config.ts` already reads every `-tables/-table.ts`, and `db/index.ts` re-exports it.
- `db/scheduling-tables/commitment-table.ts`: one row says one person or place is taken from `startsAt` to `endsAt`, by a booking or by time off, active or cancelled. Checks refuse a kind or status outside the lists and an end that is not after the start. A composite foreign key to `resource(organizationId, id)` keeps a row inside its business. It is `no action` on delete, deliberately not `restrict`: both refuse deleting a person who has any row (cancelled or past included, F-50), but deleting a business removes its people and their rows in one statement, and only `no action`, checked at the end of that statement, lets that through. `commitment_resource_index` on `(organizationId, resourceId)` serves both delete checks (F-49).
- Migration `0009_commitment.sql`: generated, then finished by hand. Its first statement switches on Postgres's standard `btree_gist` add-on (Frank's yes, 2026-10-01); its last adds `commitment_no_overlap`, the exclusion constraint Drizzle cannot express: no two active rows with the same business and person whose half-open `[startsAt, endsAt)` ranges overlap. The business is in the rule on purpose (F-48): without it, Postgres checked the overlap before the foreign key, so a row naming another business's busy person was refused as busy, leaking that person's schedule. A later `db:generate` finds no change, so it never tries to drop the rule. 0009 was edited in place during the feature, while it had only been applied to the laptop's database.

backend/lib/scheduling

- `hold-time.ts`: all of one booking's or one stretch of time off's people and places, in one multi-row `INSERT`. One statement is already all or nothing in Postgres, so no explicit transaction (the spec said one; the reviewer confirmed the single statement holds). `23P01`, the rule refusing a row, is the only refusal read as an answer (`{ held: false }`). Two holds that clash at the same instant can deadlock inside Postgres, which cancels one with `40P01`; that side tries again, up to three attempts in all, and by then the other is saved, so it answers taken (F-51). Every other error leaves only a safe reason. An empty list or a person given twice is refused before the database, since a duplicate would otherwise clash with itself and falsely answer taken.
- `release-time.ts`: marks rows cancelled, only the business's own and only active ones, and answers how many changed. The rows are kept.
- `find-commitments.ts`: the active rows of some people and places whose time overlaps a stretch, ordered by start. The overlap is written exactly as the rule writes it (`tstzrange(..., '[') && tstzrange(..., '[')`), so a row that only meets the stretch's edge is not returned and the query can use the rule's index.

- Tests: `commitment-rules.test.ts` inserts rows directly to prove every database rule by the name of the constraint that refuses; one test file per function beside it. Each test makes its own throwaway business. Every test was shown able to fail, by breaking the rule on the local database (5a.1) or planting sixteen faults one at a time (5a.2). The deadlock test runs ten clashing pairs on purpose; each deadlock costs Postgres about a second to detect, so it takes several seconds.

What later features inherit

- 5c reads `findCommitments` and subtracts it from bookable hours.
- 5d calls `holdTime` for a booking with its buffers already inside the range, and must cancel a booking and release its time together; `holdTime` and `releaseTime` cannot yet join a caller's transaction (F-52, carried).
- Three or more holds on one person at the very same instant could in principle exhaust the three attempts and end in an error rather than taken; never a double booking. Worth a line in 5c or 5d.
- Feature 12 decides what deleting a person with commitments does.
- At deploy: confirm Railway lets the migration create `btree_gist`, and apply 0005 to 0009.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live. F-32, F-47, F-52 and F-53 stay in the live ledger.

5a/F-48 [P2] closed - Another business's person answers "taken" when busy and a key error when free

File: packages/shared/migrations/0009_commitment.sql:22 **Found:** 2026-10-01 by /audit (scope: step 5a.1, 790512c..1def0b9; lens: security) **Why it matters:** The exclusion constraint is checked as the row enters its index, before the composite foreign key (an end-of-statement trigger) runs. Probed in a rolled-back transaction on `scheduleads_dev`: a row from business A naming business B's person over B's booked time fails with `23P01 commitment_no_overlap`; the same row over B's free time fails with `23503 commitment_resource_fk`. The 5a.1 test only covers the free case, so it passes. 5a.2's `holdTime` contract maps `23P01` to `{ held: false }` and anything else to an error, so holding another business's person would tell the caller when that person is busy, and Postgres's `23P01` detail even carries B's existing range. Not reachable today (no caller, and resource ids are random), but it is exactly the shape 5a.2 will be built on, and the spec's "the rule needs no business column" (current-feature.md:132) is what lets it through. **Suggested fix:** Either add `"organizationId" WITH =` as the first element of `commitment_no_overlap` in 0009 (applied only locally so far, so editing it is allowed; ids stay unique, so the rule is no weaker, and the cross-business row then always fails on the foreign key), or make 5a.2's `holdTime` confirm every resource id belongs to the business before inserting. Either way add a test: holding another business's busy person answers the same as a free one, never "taken". Correct the spec line at 132. **Resolution:** Fixed 2026-10-01 in 5a.1's review-fix commit: `commitment_no_overlap`

now compares `"organizationId"` first, edited into 0009 (local only). A new test holds another business's busy person and expects `23503 commitment_resource_fk`; it failed with `23P01` against the old rule and passes now. Spec corrected. Closed 2026-10-01 by the independent review of 1def0b9..d5175ae: the live `scheduleads_dev` constraint reads `EXCLUDE USING gist ("organizationId" WITH =, "resourceId" WITH =, tstzrange(...)) WITH &&) WHERE status = 'active'`, the ledger row's hash matches the edited 0009, and both the 5a.1 rule test and 5a.2's `holdTime` test (another business's person, busy and free, both `23503`, nothing held) pass. Ids stay globally unique, so the rule is no weaker. No new defect from the repair.

5a/F-49 [P3] closed - Nothing indexes commitment by business and person, so its two foreign key checks scan the whole table

File: packages/shared/db/scheduling-tables/commitment-table.ts:34 **Found:** 2026-10-01 by /audit (scope: step 5a.1, 790512c..1def0b9; lens: performance) **Why it matters:** Every other table here that points at a parent through `(organizationId, x)` has an index leading with those columns (`availability_rule_resource_unique`, `activity_timeline_index`). `commitment` has only its primary key and the exclusion constraint's GiST index, which is partial (`where status = 'active'`), so it cannot serve the foreign key checks. `EXPLAIN` with sequential scans disabled still picks a sequential scan for both `"organizationId" = $1 and "resourceId" = $2` (the check run on every person deleted) and `"organizationId" = $1` (the cascade when a business is deleted). This is the table that grows with every booking, forever. Rare operations, so small today. **Suggested fix:** `index("commitment_resource_index").on(table.organizationId, table.resourceId)` in the table and the migration, in 5a.1's migration while it is local only, or with 5a.2. **Resolution:** Fixed 2026-10-01: `commitment_resource_index` on `(organizationId, resourceId)` in the table and 0009. Not proved by a test (an index changes speed, not answers). Closed 2026-10-01 by the independent review of 1def0b9..d5175ae: the index exists on `scheduleads_dev`, `db:generate` reports no change, and `EXPLAIN` with sequential scans off now gives an index-only scan on `commitment_resource_index` for both the person check and the business cascade. No new defect.

5a/F-50 [P3] closed - "A person who still holds time cannot be deleted" is really "a person who ever held time"

File: packages/shared/db/scheduling-tables/commitment-table.ts:34 **Found:** 2026-10-01 by /audit (scope: step 5a.1, 790512c..1def0b9; lens: quality) **Why it matters:** The table comment and the spec (current-feature.md:117) say a person or place that still holds time cannot be deleted. The foreign key ignores `status` and time, so a cancelled row, or a booking finished years ago (status stays `active`, there is no "done"), blocks the delete just the same. Feature 12 decides what deleting a person does, and will read this wording and expect a person with only past or cancelled bookings to be deletable. **Suggested fix:** Reword both to "any commitment row, cancelled or past included, keeps the person or place from being deleted", and add one line on feature 12 in `build-plan.md` so the choice (deactivate instead, or remove the rows first) is made there knowingly. **Resolution:** Fixed 2026-10-01: the table comment and the spec say any row, cancelled or past included, blocks the delete; the delete test now

also covers a person with only a cancelled row; feature 12 in build-plan.md carries the decision. Closed 2026-10-01 by the independent review of 1def0b9..d5175ae: the table comment (commitment-table.ts:35-36), the spec's Data / contracts and build-plan.md item 12 all say any row, cancelled or past included; the delete test loops over a person with an active row and one with only a cancelled row and expects 23503 for both. No stale "still holds time" wording remains. No new defect.

5a/F-51 [P1] closed - Two holds at the same instant can deadlock, and the loser gets an error instead of "taken"

File: backend/lib/scheduling/hold-time.ts:51 **Found:** 2026-10-01 by /audit independent (scope: step 5a.2, 1def0b9..d5175ae; lens: quality, tests) **Why it matters:** An exclusion constraint does not serialize concurrent inserts the way a unique index does. Each statement puts its row into the GiST index first and then looks for conflicts; when two statements do that at the same moment, each sees the other's uncommitted row, each waits for the other, and Postgres's deadlock detector kills one with 40P01 after deadlock_timeout (1 second). holdTime maps only 23P01 to { held: false }, so the killed one throws Holding time failed: database error 40P01. Probed on scheduleads_dev with holdTime's exact statement shape (one multi-row INSERT, seeded clinic-dev resources, every transaction rolled back, nothing left): 400 simultaneous pairs on the same one resource gave 51 deadlocks; 400 pairs on the same two resources in opposite order gave 121. That breaks step 5a.2's Done when ("two holds for the same time at the same instant give one held and one taken"), the claim at hold-time.ts:2-3, and the spec's "Concurrent inserts are serialized by Postgres itself: the second waits, then fails" (current-feature.md:136). The saved race test passed 25 of 25 reruns because its two calls rarely reach the database in the same microseconds, so it cannot see this. No double booking results; a customer who lost the race sees a failure instead of "that time was just taken". Sorting the resource ids does not help: the single-resource case deadlocks too. **Suggested fix:** In holdTime's catch, retry the same insert on 40P01 (two or three attempts are plenty): by then the survivor has committed, so the retry gets a clean 23P01 and answers taken, or succeeds if the survivor was rolled back. Add a test that runs many simultaneous pairs (for example 100) and expects every result to be held or taken, never thrown, and show it failing without the retry. Correct hold-time.ts:2-3 and current-feature.md:136.

Resolution: Fixed 2026-10-01: holdTime tries a 40P01 again, up to three attempts in all; the header comment and the spec now say so. A new test runs ten simultaneous pairs on one person and one room in opposite order and expects every pair to answer one held, never an error. With the retry removed it failed five runs in five with 40P01; with it, it passes (seven to nine seconds, as each deadlock costs Postgres one second to detect). Reproduced first with the real holdTime: 150 pairs failed three runs in three. Closed 2026-10-01 by the independent review of c01dd9c..da22890: holdTime (hold-time.ts:38-63) retries only 40P01, at most three attempts, with fresh row ids each time, and a deadlock victim is a single autocommitted statement, so nothing of it was saved before the retry. 23P01 still answers { held: false } and every other error still leaves only the safe reason. The stress test ran 6.1 s in this review's run, so several real deadlocks happened and were all answered held or taken; the full backend suite (180 tests) passes. The header comment and the spec's Data / contracts now describe the retry. No new defect in the repaired behaviour; the finding number the repair put in the test's comment is a separate standards

finding, F-53. Three or more simultaneous holds on one person can in principle deadlock more than twice; noted as remaining risk in review.md, not a defect against 5a's two-hold contract.

Independent review

Status: passed **Target commit:** da22890dffa20345f3b09440d5aa0f828e881162 **Base commit:** c01dd9c8647a2533232155f4fea752bd95bd9c07 **Base ref:** main **Spec hash:** b1ec992edce1edeb6280d9a63414cad4b3797e55c68c6556e435712e4ab4dd1a **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-01T21:14:23Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-01T21:18:16Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`c01dd9c8647a2533232155f4fea752bd95bd9c07..da22890dffa20345f3b09440d5aa0f828e881162` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `git rev-parse HEAD`, `git merge-base main HEAD`, `sha256sum blueprint/context/current-feature.md`, `git status --porcelain --untracked-files=all` : pass (target, base and spec hash match; only review.md differs)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass
- `npm run test --workspace=@scheduleads-app/shared` : pass (9 files, 66 tests)
- `npm run test --workspace=backend` : pass (15 files, 180 tests)
- `npx vitest run lib/scheduling/ --reporter=verbose` (in `backend`) : pass (4 files, 28 tests; the deadlock stress test took 6.1 s)
- `npm run format:check` : pass
- `npm run db:generate --workspace=@scheduleads-app/shared` : pass ("No schema changes, nothing to migrate", no files written)
- Read-only queries and a rolled-back `EXPLAIN` against local `scheduleads_dev` : pass

Evidence

- Whole delta read: the commitment table, 0009 migration, journal and snapshot entry, `holdTime`, `releaseTime`, `findCommitments`, their four test files, and the spec, build-plan, project-plan and overview changes.
- Live `commitment_no_overlap` on `scheduleads_dev` is `EXCLUDE USING gist ("organizationId" WITH =, "resourceId" WITH =, tstzrange("startsAt", "endsAt", '[]')) WITH &&) WHERE status = 'active'`; `commitment_resource_fk` has no delete action; `btree_gist` 1.8 is installed.
- The applied 0009 row in `drizzle.__scheduleads_app_migrations` has the same SHA-256 as the committed `0009_commitment.sql`, so the database holds exactly the reviewed migration. The commitment table is empty after the test runs (tests clean up).
- `findCommitments` ' query, under a rolled-back `enable_seqscan = off`, plans as an index scan on `commitment_resource_index`; its time bounds are bound parameters, not string-built SQL.
- Every Done when of 5a.1 and 5a.2 has a test that names the refusing constraint or the exact answer; release and find filter by business, hold relies on the composite foreign key and is proved refused as `23503` for another business's busy and free person.
- F-51 re-examined: the retry path ran (several one-second deadlocks in the 6.1 s stress test) and every pair answered held or taken.
- No em dashes in the delta; one finding number left in a test comment (F-53).

Findings

- F-51 [P1] closed by this review (re-examined, defect gone, no new defect in the repaired behaviour).
- F-53 [P3] open, new: a finding number in the deadlock test's comment (backend/lib/scheduling/hold-time.test.ts:111).
- F-52 [P3] open, carried to 5d unchanged.
- No P0 or P1 open or fixed.

Remaining risk

- `holdTime` tries a deadlock at most three times. Two holds on the same time resolve within that; three or more simultaneous holds on the same person can in principle deadlock more than twice, and the last one would then throw instead of answering taken. Not reproduced (probing needs committed rows, outside this review's limits). No double booking either way; worth one line in 5c or 5d if the widget can fire several holds at once.
- The spec's Testing section cites "F-53 and F-54" bare, meaning feature 4's 4/F-53 and 4/F-54; the live ledger now has its own F-53. Write them prefixed if the spec is touched again.
- The dashboard activity helper (`run-state.mjs`) was not run: this reviewer was limited to writing review.md and findings.md.

- Check was not required and was not run; there is no browser or route surface in 5a.